Announcements:

> 公告：

- Join the CS336 slack

> 加入 CS336 的 Slack

- Sign up on Modal with your **Stanford** email

> 用你的 Stanford 邮箱注册 Modal

- Read the AI policy guide

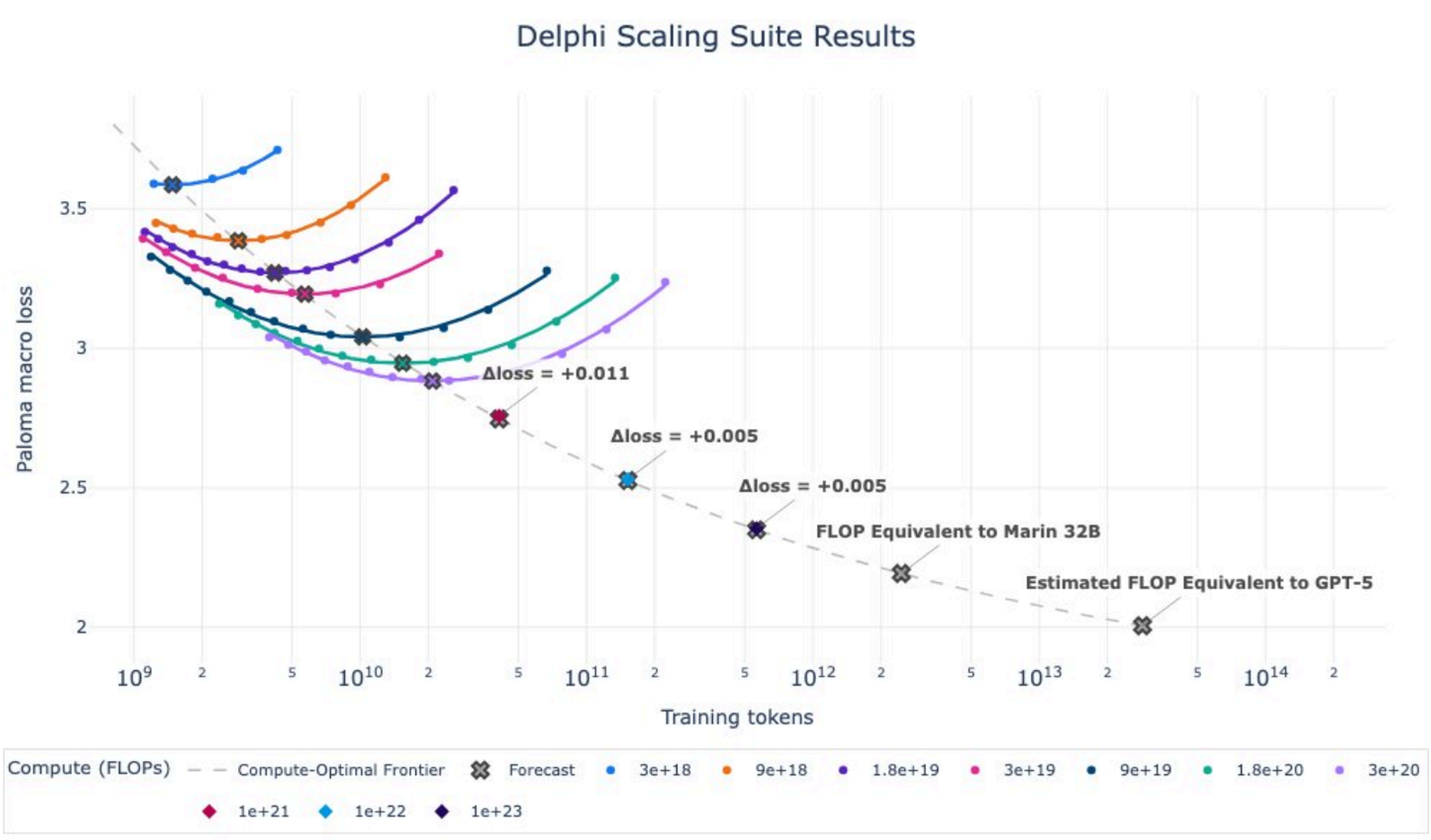
> 阅读 AI 政策指南

- Read the cluster guide

> 阅读集群指南

Marin 1e23 FLOPs run finished and matched forecasts!

> Marin 的 1e23 FLOPs 训练跑完了，而且与预测相符！



Last lecture: overview, tokenization

> 上一讲：概览、分词（tokenization）

Today: resource accounting (systems)

> 今天：资源核算（系统部分）

Recall: what's the best model one can train given fixed resources (compute, memory)?

> 回顾：在给定资源（算力、内存）下，我们能训练出的最好模型是什么？

In other words: maximize (computational) **efficiency**.

> 换言之：最大化（计算）效率。

Prerequisite: understand the resources (compute, memory) for a given computation.

> 前提条件：理解一个给定计算所需的资源（算力、内存）。

What knowledge to take away from this lecture:

> 从本讲中应带走的知识：

- Mechanics: straightforward (PyTorch semantics)

> 具体机制：简单直接（PyTorch 语义）

- Mindset: resource accounting (remember to do it)

> 思维方式：资源核算（记得要做）

- Intuitions: get a sense of how resources are spent, no ML magic today

> 直觉：建立对资源如何被消耗的感觉，今天没有 ML 魔法

Summary:

> 总结：

- Everything is operations on tensors (parameters, gradients, activations, optimizer states, data)

> 一切皆是对张量的操作（参数、梯度、激活值、优化器状态、数据）

- einops: better way to think about tensor operations

> einops：一种思考张量操作的更好方式

- 6 (# data points) (# parameters) FLOPs per training step

> 每个训练步 6 (# 数据点) (# 参数) FLOPs

- Arithmetic intensity / roofline analysis: compute-bound or memory-bound?

> 算术强度 / roofline 分析：计算受限还是访存受限?

- Matrix multiplications are compute-bound, elementwise operations are memory-bound

> 矩阵乘法是计算受限的，逐元素（elementwise）运算是访存受限的

- Gradient accumulation, activation checkpointing: reduce memory to use bigger batch sizes

> 梯度累积、激活检查点：减少内存以使用更大的 batch size

Question: How long would it take to train a 70B parameter model on 15T tokens on 1024 H100s?

> 问题：在 1024 张 H100 上，用 15T token 训练一个 70B 参数的模型需要多久?

Question: What's the largest model that can you can train on 8 H100s using AdamW?

> 问题：在 8 张 H100 上用 AdamW 能训练的最大模型是多大?

Caveat: activations are not accounted for (depends on batch size and sequence length), so this is an upper bound.

> 提醒：激活值未计入（取决于 batch size 和序列长度），所以这是一个上界。

This is a rough back-of-the-envelope calculation.

> 这只是一个粗略的随手估算。

But it gives you the flavor of napkin math one can quickly do to get a sense of resources.

> 但它能让你体会那种可以快速上手的餐巾纸数学（napkin math），从而对资源开销建立感觉。

Tensors are the basic building block for storing everything:

> 张量是存储一切内容的基本构件：

- data

> 数据

- parameters

> 参数

- gradients

> 梯度

- optimizer state

> 优化器状态

- activations

> 激活值

Example: parameters of the DeepSeek v3.2 model [DeepSeek-AI+ 2025]

> 示例：DeepSeek v3.2 模型的参数 [DeepSeek-AI+ 2025]

[DeepSeek v3.2 model on Hugging Face]

> [Hugging Face 上的 DeepSeek v3.2 模型]

Each tensor has a rank, which is the number of dimensions.

> 每个张量都有一个秩（rank），即维度的个数。

In Transformers, will see tensors of rank 4:

> 在 Transformer 中会见到秩为 4 的张量：

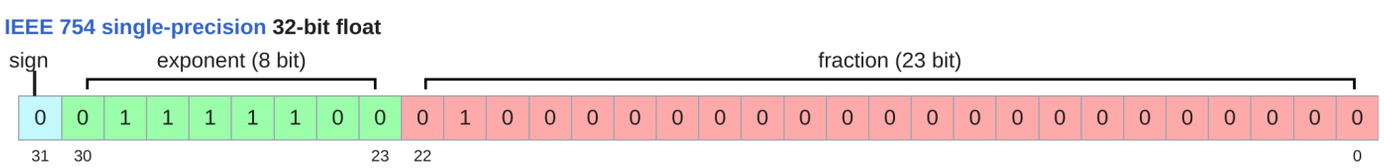
Elements of tensors are generally floating point numbers.

> 张量的元素一般是浮点数。

fp32

[Wikipedia]

> [维基百科]



The fp32 data type (also known as float32 or single precision) is the default.

> fp32 数据类型（又称 float32 或单精度）是默认类型。

Traditionally, in scientific computing, fp32 is the baseline; you could use double precision (fp64) in some cases.

> 传统上，科学计算以 fp32 为基线；某些情况下可以使用双精度（fp64）。

In deep learning, you can be a lot sloppier.

> 在深度学习中，要求可以宽松得多。

Let's examine memory usage of these tensors.

> 下面来看这些张量的内存占用。

Memory is determined by the (i) number of values and (ii) data type of each value.

> 内存由 (i) 数值的个数和 (ii) 每个数值的数据类型决定。

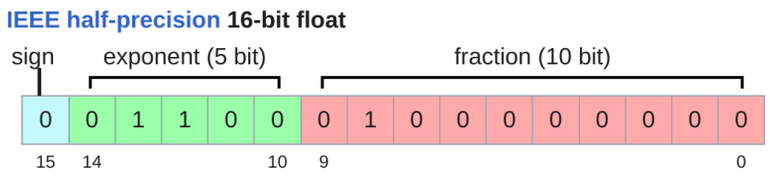
One matrix in the feedforward layer of GPT-3:

> GPT-3 前馈层中的一个矩阵：

fp16

[Wikipedia]

> [维基百科]



The fp16 data type (also known as float16 or half precision) cuts down the memory.

> fp16 数据类型（又称 float16 或半精度）削减了内存占用。

However, the dynamic range (especially for small numbers) isn't great.

> 但它的动态范围（尤其是对小数值）不太好。

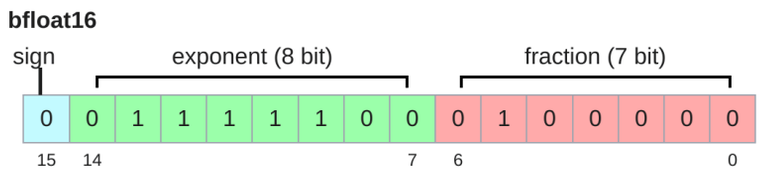
If this happens when you train, you can get instability.

> 如果训练时出现这种情况，就可能引发不稳定。

bf16

[Wikipedia]

> [维基百科]



Google Brain developed brain floating point (bf16) in 2018 to address this issue.

> Google Brain 于 2018 年开发了 brain floating point（bf16）来解决这一问题。

bf16 uses the same memory as fp16 but has the same dynamic range as fp32!

> bf16 与 fp16 占用相同的内存，却具有与 fp32 相同的动态范围！

The only catch is that the resolution is worse, but this matters less for deep learning.

> 唯一的缺点是分辨率更低，但对深度学习影响不大。

Mixed precision

> 混合精度

Implications on training:

> 对训练的影响：

- Training with fp32 works, but requires lots of memory.

> 用 fp32 训练可行，但内存开销很大。

- Training with fp16 and even bf16 is risky, and you can get instability.

> 用 fp16 甚至 bf16 训练都有风险，可能出现不稳定。

Solution: mixed precision training [Micikevicius+ 2017]

> 解决方案：混合精度训练 [Micikevicius+ 2017]

- Use bf16 for parameters, activations, and gradients

> 参数、激活值和梯度使用 bf16

- Use fp32 for optimizer states

> 优化器状态使用 fp32

Pytorch has an automatic mixed precision (AMP) library. [docs]

> PyTorch 提供了自动混合精度（AMP）库。 [文档]

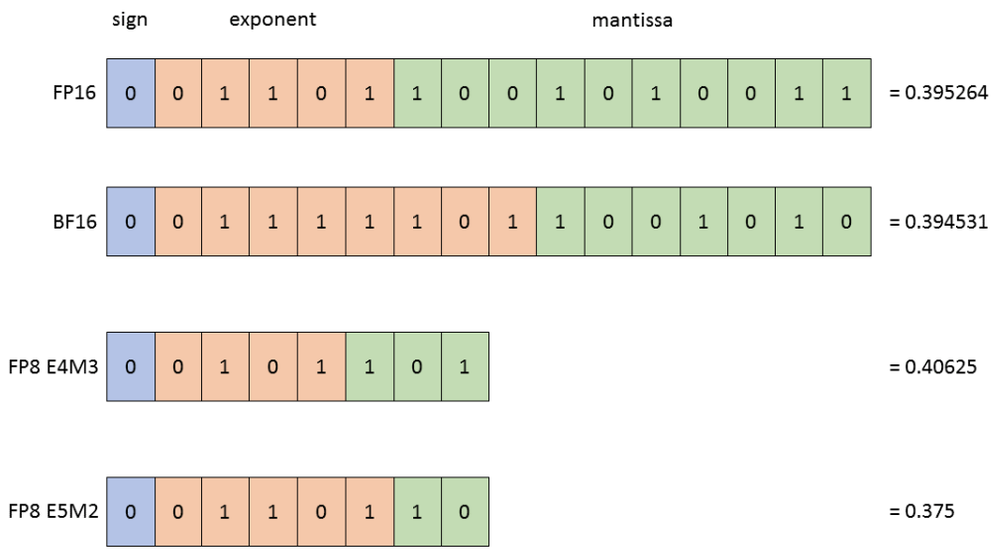
Tries to cast things into bf16 when safe (matmuls, not exp).

> 在安全的情况下会把运算转换成 bf16（比如 matmul，而 exp 不会）。

fp8

In 2022, fp8 was standardized, motivated by machine learning workloads primer.

> 2022 年，fp8 完成标准化，动机来自机器学习工作负载。



H100s support two variants of FP8: E4M3 (range [-448, 448]) and E5M2 ([-57344, 57344]).

> H100 支持两种 FP8 变体：E4M3（范围 [-448, 448]）和 E5M2（[-57344, 57344]）。

Reference: [Micikevicius+ 2022]

> 参考文献： [Micikevicius+ 2022]

fp4

In 2025, NVIDIA developed nvfp4

> 2025 年，NVIDIA 开发了 nvfp4

Only 4 bits per value!

> 每个值只占 4 bit！

Values: -6, -4, -3, -2, -1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 1.5, 2, 3, 4, 6

> 可取的值：-6, -4, -3, -2, -1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 1.5, 2, 3, 4, 6

Use a separate scale factor per block, so actually get more dynamic range (but just can't vary freely from neighbors).

> 每个块使用单独的缩放因子，因此实际上能获得更大的动态范围（只是无法相对邻近的值自由变化）。

Nemotron 3 Super was trained in NVFP4

[Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning]

> Nemotron 3 Super 使用 NVFP4 训练 [Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning]

Some of this is done in NVIDIA libraries outside of user control.

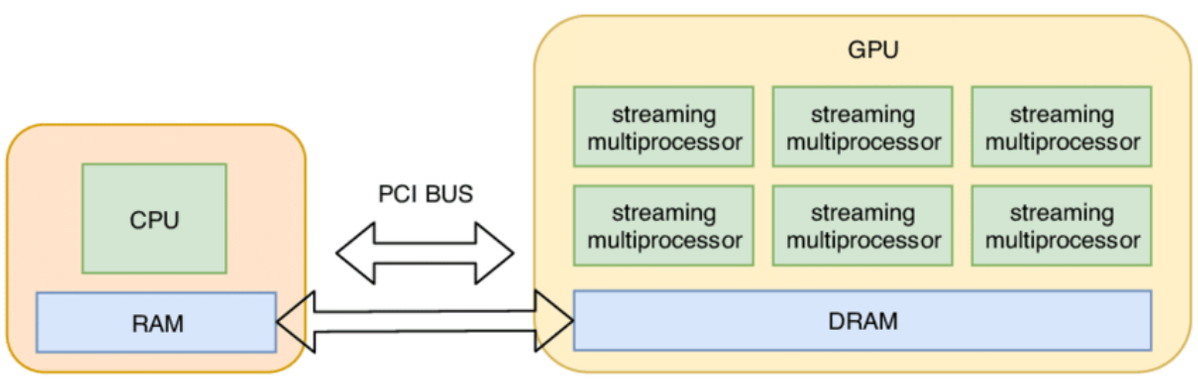
> 其中一部分是在 NVIDIA 库里完成的，不受用户控制。

By default, tensors are stored in CPU memory.

> 默认情况下，张量存储在 CPU 内存中。

However, what about GPUs?

> 那 GPU 呢?



In order to take advantage of the massive parallelism of GPUs, we need to move them to GPU memory.

> 为了利用 GPU 的大规模并行能力，我们需要把张量搬到 GPU 显存。

Or create the tensor directly on the GPU:

> 或者直接在 GPU 上创建张量：

Einops is a library for manipulating tensors where dimensions are named.

> Einops 是一个按命名维度来操作张量的库。

It is inspired by Einstein summation notation (Einstein, 1916).

> 它受爱因斯坦求和记号（Einstein, 1916）启发。

[Einops tutorial]

> [Einops 教程]

Traditional PyTorch code:

> 传统 PyTorch 写法：

Easy to mess up the dimensions (what is -2, -1?)...

> 维度很容易搞混（-2、-1 是什么来着？）……

Einsum is generalized matrix multiplication with good bookkeeping.

> einsum 是一种带清晰维度记账的广义矩阵乘法。

Let's try a more complex example...

> 来试试一个更复杂的例子……

Dimensions that are not named in the output are summed over.

> 没有在输出中命名的维度会被求和掉。

You can reduce a single tensor via some operation (e.g., sum, mean, max, min).

> 可以通过某个操作（如 sum、mean、max、min）对单个张量进行归约。

Sometimes, a dimension represents two dimensions

> 有时，一个维度其实代表两个维度

...and you want to operate on one of them.

> ……而你只想对其中的一个进行操作。

...where `total_hidden` is a flattened representation of `heads * hidden1`

> ……其中 total_hidden 是 heads * hidden1 展平后的表示

Having gone through all the operations, let us examine their computational cost.

> 过完了所有这些操作，我们来考察它们的计算开销。

A floating-point operation (FLOP) is a basic operation like addition (x + y) or multiplication (x y).

> 浮点运算（FLOP）指加法（x + y）或乘法（x y）这样的基本运算。

Two terribly confusing acronyms (pronounced the same!):

> 两个极易混淆的缩写（读音还相同！）：

- FLOPs: floating-point operations (measure of computation done)

> FLOPs：浮点运算数（衡量完成的计算量）

- FLOP/s: floating-point operations per second (also written as FLOPS), which is used to measure the speed of hardware.

> FLOP/s：每秒浮点运算数（也写作 FLOPS），用于衡量硬件速度。

Intuitions

> 直觉

Training GPT-3 (2020) took 3.14e23 FLOPs. [article]

> 训练 GPT-3（2020）耗费 3.14e23 FLOPs。 [文章]

Training GPT-4 (2023) is speculated to take 2e25 FLOPs. [article]

> 训练 GPT-4（2023）据推测需要 2e25 FLOPs。 [文章]

H100 has a peak performance of 1979 teraFLOP/s with sparsity, 50% without [spec]

> H100 的峰值性能为 1979 teraFLOP/s（含稀疏），不含稀疏为 50% [规格表]

8 H100s for 2 weeks:

> 8 张 H100 跑两周：

Linear model

> 线性模型

How many FLOPs is this matmul?

> 这个 matmul 是多少 FLOPs?

We have one multiplication (x[i][j] * w[j][k]) and one addition per (i, j, k) triple.

> 每个 (i, j, k) 三元组对应一次乘法 (x[i][j] * w[j][k]) 和一次加法。

We can also time this operation to see how long it takes.

> 我们还可以给这个操作计时，看看它要跑多久。

The actual FLOP/s of this operation:

> 该操作的实际 FLOP/s：

Each GPU has a specification sheet that provides the peak performance.

> 每款 GPU 都有一张规格表，标明峰值性能。

- Example: [H100 spec]

> 示例： [H100 规格表]

Note that the FLOP/s depends heavily on the data type!

> 注意 FLOP/s 在很大程度上取决于数据类型！

Model FLOPs utilization (MFU)

> 模型 FLOPs 利用率（MFU）

Definition: MFU = (actual FLOP/s) / (promised FLOP/s) [ignore communication/overhead]

> 定义：MFU = (实际 FLOP/s) / (标称 FLOP/s)（忽略通信/开销）

Usually, MFU of ≥ 0.5 is quite good!

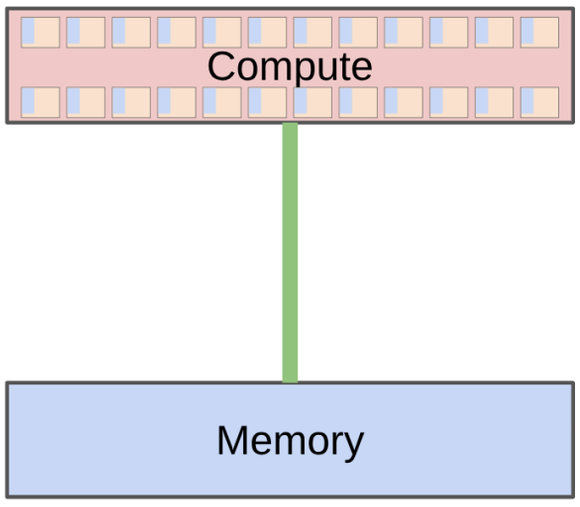
> 通常 MFU ≥ 0.5 就相当不错了！

But why is MFU not closer to 1?

> 但为什么 MFU 不能更接近 1 呢?

To answer this question, we need to look more closely at how computations are done on GPUs...

> 要回答这个问题，需要更细致地考察 GPU 上的计算是如何执行的……



How to compute a thing:

> 如何计算一个东西：

1. Send inputs from memory to accelerator

> 把输入从内存送到加速器

2. Perform computation

> 执行计算

3. Send outputs from accelerator to memory

> 把输出从加速器送回内存

How long does this take?

> 这需要多长时间?

Depends on two things:

> 取决于两件事：

1. Accelerator speed (FLOP/s)

> 加速器速度（FLOP/s）

2. Memory bandwidth (bytes/s)

> 内存带宽（bytes/s）

Assume we can overlap communication and computation perfectly.

> 假设我们能把通信和计算完美重叠。

What is the bottleneck?

> 瓶颈是什么?

- Memory-bound: communication time > computation time

> 访存受限：通信时间 > 计算时间

- Compute-bound: computation time > communication time

> 计算受限：计算时间 > 通信时间

In this case, ReLU is memory-bound.

> 在这个例子中，ReLU 是访存受限的。

Alternative way to see this:

> 换一种方式来看：

Accelerator intensity: how much work can the accelerator do per byte transferred?

> 加速器强度：每传输一个字节，加速器能完成多少工作?

Arithmetic intensity: how much actual work per byte for this workload?

> 算术强度：这个工作负载每字节对应多少实际工作量?

What is the bottleneck?

> 瓶颈是什么?

- Memory-bound: arithmetic intensity < accelerator intensity

> 访存受限：算术强度 < 加速器强度

- Compute-bound: arithmetic intensity > accelerator intensity

> 计算受限：算术强度 > 加速器强度

In general, we'll find ourselves memory bound.

> 一般而言，我们会发现自己常处于访存受限的状态。

Can we increase arithmetic intensity?

> 能提高算术强度吗?

Note that GeLU does more work than ReLU per byte moved, so it has higher arithmetic intensity.

> 注意，GeLU 每搬运一个字节所做的工作比 ReLU 多，因此它的算术强度更高。

But still memory-bound!

> 但仍然是访存受限!

In other words, ReLU is not faster than GeLU (when doing things in an isolated way).

> 换句话说，（单独执行时）ReLU 并不比 GeLU 快。

Memory-bound!

> 访存受限!

Memory-bound!

> 访存受限!

Finally, compute-bound!

> 终于计算受限了!

As long as we have large matrices, we're compute-bound (saturating the accelerator).

> 只要矩阵足够大，我们就是计算受限的（能吃满加速器）。

Training Transformers involves big matrix multiplications.

> 训练 Transformer 涉及大型矩阵乘法。

Matrix-vector product is what happens during inference, which is why inference is memory-bound.

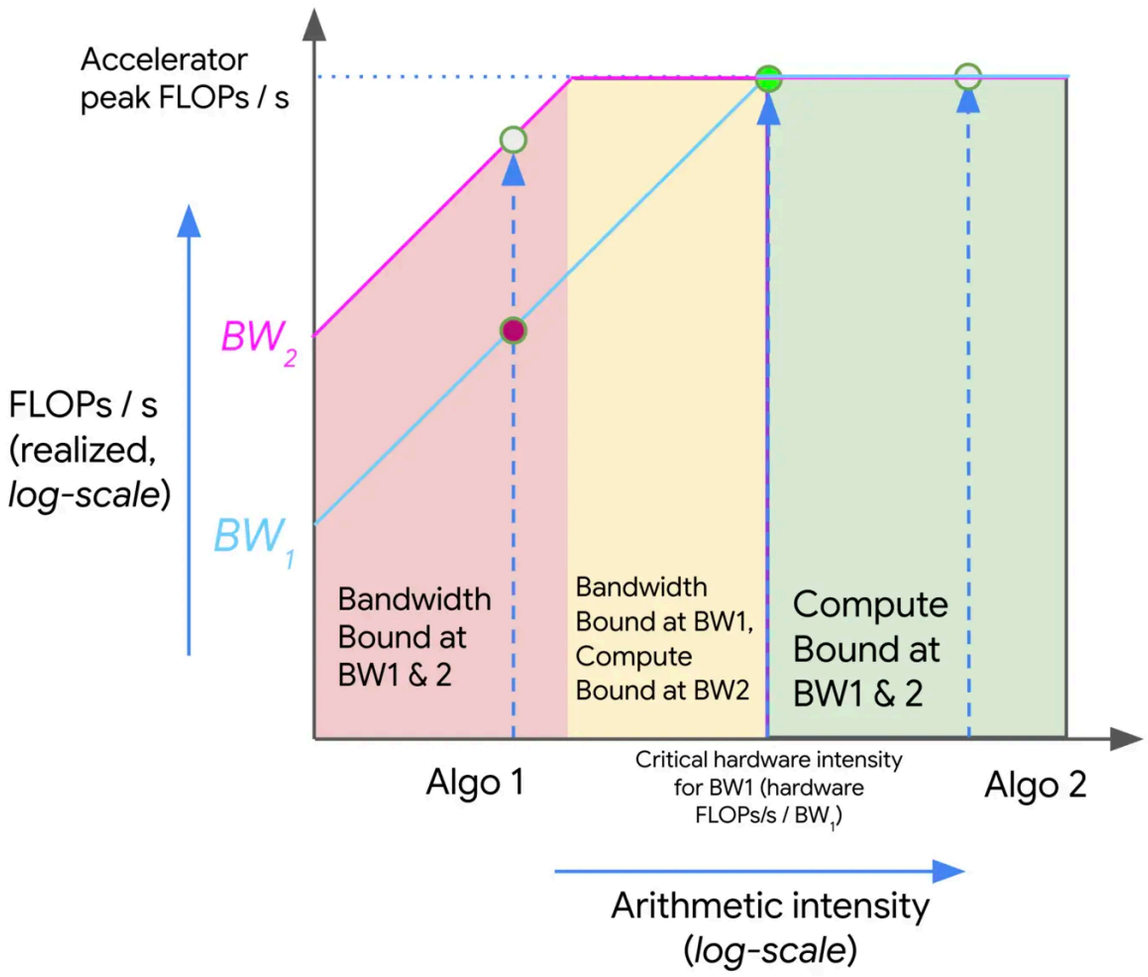
> 推理阶段发生的是矩阵-向量乘法，这正是推理访存受限的原因。

Note: arithmetic/accelerator intensity also depends on the precision (bf16 versus fp32).

> 注意：算术强度/加速器强度还取决于精度（bf16 与 fp32）。

We can visualize the relationship between arithmetic intensity and performance using roofline plots.

> 我们可以用屋顶线（roofline）图来可视化算术强度与性能之间的关系。



- Each slice on the x-axis is a particular computation (with some arithmetic intensity)

> x 轴上的每一小段是一种特定的计算（具有某个算术强度）

- Each piecewise linear function corresponds to a particular hardware

> 每条分段线性函数对应一种特定硬件

- Kink is the accelerator intensity (transition from memory-bound to compute-bound)

> 拐点即加速器强度（从访存受限到计算受限的转折）

We can now relate this back to MFU:

> 现在可以把它与 MFU 联系起来：

MFU = min(1, arithmetic-intensity / accelerator-intensity)

> MFU = min(1, 算术强度 / 加速器强度)

[reference]

> [参考文献]

So far, we've constructed tensors and passed them through operations (forward).

> 到目前为止，我们构建了张量，并把它们送入各种操作（前向）。

Now, we're going to compute the gradient (backward).

> 现在，我们要计算梯度（反向）。

As a simple example, let's consider the simple linear model:

> 作为一个简单例子，考虑下面这个简单的线性模型：

y = 0.5 (x * w - 5)^2
Forward pass: compute loss

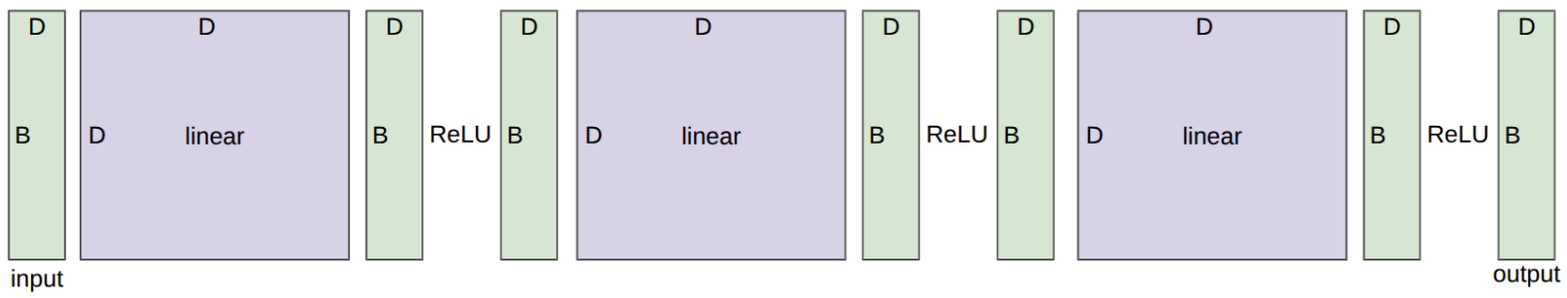
> 前向传播：计算损失

Backward pass: compute gradients

> 反向传播：计算梯度

Let us count the FLOPs for computing gradients.

> 我们来数一数计算梯度所需的 FLOPs。



Define a simplified model (2-layer linear network):

> 定义一个简化模型（2 层线性网络）：

Zoom in on one layer

> 放大观察其中一层

Let's focus on the second layer (h2 = h1 @ w2)

> 聚焦第二层（h2 = h1 @ w2）

Forward pass: Recall the number of forward FLOPs:

> 前向传播：回顾前向的 FLOPs 数量：

Backward pass: How many FLOPs is running the backward pass?

> 反向传播：跑一遍反向传播需要多少 FLOPs?

We need to compute:

> 我们需要计算：

- h1.grad = d loss / d h1
- w2.grad = d loss / d w2

Note that the backward pass is 2x more expensive than the forward pass.

> 注意，反向传播的开销是前向传播的 2 倍。

Consider all layers

> 考虑所有层

This was just for w2, need to apply it to all parameters in the network.

> 上面只是针对 w2 的，还需要对网络中的所有参数做同样的核算。

Putting it together:

> 综合起来：

- Forward pass: 2 (# data points) (# parameters) FLOPs

> 前向传播：2 (# 数据点) (# 参数) FLOPs

- Backward pass: 4 (# data points) (# parameters) FLOPs

> 反向传播：4 (# 数据点) (# 参数) FLOPs

- Total: 6 (# data points) (# parameters) FLOPs

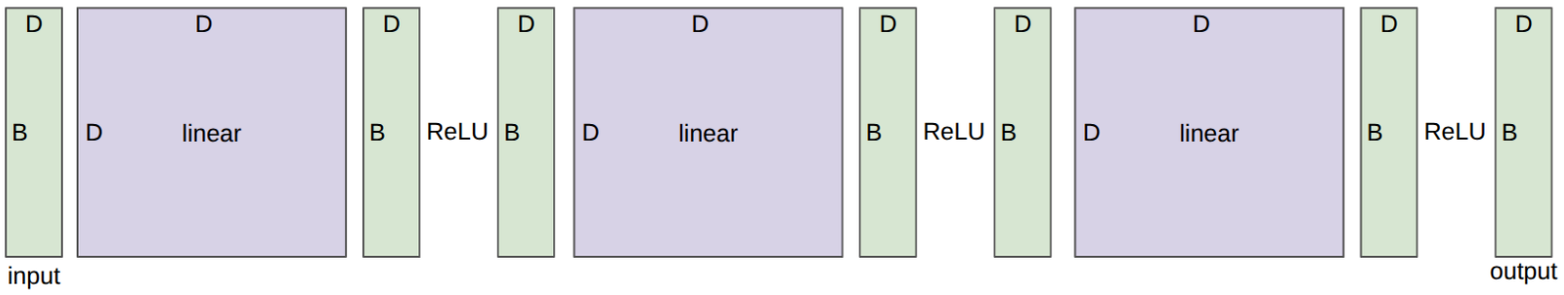
> 总计：6 (# 数据点) (# 参数) FLOPs

This is for multilayer perceptrons (MLPs)

> 这是针对多层感知机（MLP）的结果

...but it turns out to be a good approximation for Transformers for short context lengths as well.

> ……但事实证明，对短上下文长度的 Transformer 来说，这也是一个很好的近似。



Consider a deep network with L layers and D-dimensional inputs, activations, and outputs.

> 考虑一个 L 层的深层网络，其输入、激活值和输出都是 D 维的。

Recall our deep network.

> 回顾我们的深层网络。

Let's define the AdaGrad optimizer

> 我们来定义 AdaGrad 优化器

- momentum = SGD + exponential averaging of grad

> momentum = SGD + 对梯度做指数平均

- AdaGrad = SGD + averaging by grad^2

> AdaGrad = SGD + 按 grad^2 做平均

- RMSProp = AdaGrad but with exponential averaging of grad^2

> RMSProp = AdaGrad，但改为对 grad^2 做指数平均

- Adam = RMSProp + momentum

> Adam = RMSProp + momentum

AdaGrad [Duchi+ 2011]

> AdaGrad [Duchi+ 2011]

Memory

> 内存

It is customary to use fp32 for stability (accumulating averages over powers over many steps).

> 出于稳定性考虑，惯例是使用 fp32（这些平均量要在很多步上累积）。

Optimizer state memory:

> 优化器状态的内存：

- AdaGrad: 4 bytes/parameter for storing second moments

> AdaGrad：每参数 4 字节，用于存储二阶矩

- Adam: 8 bytes/parameter for storing first and second moments

> Adam：每参数 8 字节，用于存储一阶矩和二阶矩

Compute (for one training step)

> 计算量（单个训练步）

Transformers

> Transformer

The accounting for a Transformer is more complicated, but the same idea.

> 对 Transformer 做这种核算更复杂，但思路相同。

Assignment 1 will ask you to do that.

> 作业 1 会让你做这件事。

Blog post describing memory usage for Transformer training [article]

> 一篇讲述 Transformer 训练内存占用的博客 [文章]

Blog post describing FLOPs for a Transformer: [article]

> 一篇讲述 Transformer FLOPs 的博客： [文章]

Large batch sizes: improve training stability

> 大 batch size：提升训练稳定性

However, activation memory scales with batch size, so might run out.

> 但激活内存随 batch size 增长，可能会不够用。

Gradient accumulation:

> 梯度累积：

- Compute gradient on micro batches

> 在微批次（micro batch）上计算梯度

- Accumulate the gradients (don't zero it out)

> 累积梯度（不要清零）

- Every batch_size / micro_batch_size steps, update the parameters and zero out the gradients

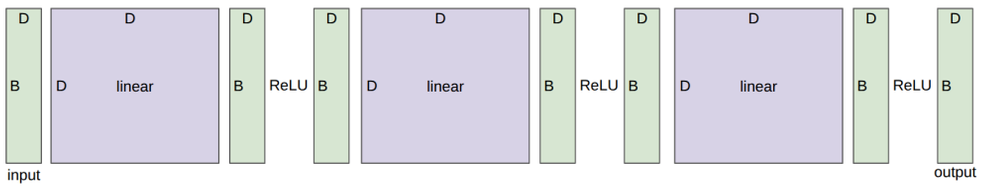
> 每 batch_size / micro_batch_size 步更新一次参数，并将梯度清零

For training, we need to store the activations of all layers

> 训练时需要存储所有层的激活值

For inference, we don't compute gradients, so we only need to store the current layer's activations.

> 推理时不计算梯度，因此只需存储当前层的激活值。



The memory usage is

> 内存占用为

Can we reduce this?

> 能减少这部分开销吗?

Activation checkpointing = gradient checkpointing = rematerialization

> 激活检查点 = 梯度检查点 = 重计算（rematerialization）

Key idea:

> 核心思想：

- Forward pass: keep only activations at subset of layers

> 前向传播：只保留一部分层的激活值

- Backward pass: recompute the missing activations from the last checkpoint

> 反向传播：从最近的检查点重算缺失的激活值

Philosophy: tradeoff memory for compute

> 理念：以计算换内存

Can we reduce this even more, especially for deep networks (large L)?

> 还能进一步减少吗，尤其是对深层网络（L 很大）？

How frequently to checkpoint?

> 检查点设多频繁?

- If store each layer's activations, then activation memory is O(L) and no recomputation.

> 若存储每一层的激活值，则激活内存为 O(L)，且无需重算。

- If store no activations, then activation memory is O(1) and compute is O(L^2) (recompute from the start for each layer).

> 若不存储激活值，则激活内存为 O(1)，计算量为 O(L^2)（每层都要从头重算）。

- If store every sqrt(L) layers, then activation memory is O(sqrt(L)) and O(L) recomputation.

> 若每 sqrt(L) 层存储一次，则激活内存为 O(sqrt(L))，重算开销为 O(L)。